



Devoir 2

Architectures Logicielles à Objets (ALO)

Document de Décisions de Conception et Justifications

Master 1 Informatique Avancée et Application (I2A)

Université Marie & Louis Pasteur, Franche-Comté

N'jie Zamon

Formation à distance en alternance
Depuis la Martinique

17 novembre 2025



Table des matières

1	Introduction et Consignes	2
2	Justification des Design Patterns Appliqués	2
2.1	Évolution 1 : Événements (Décorateur)	2
2.2	Évolution 2 : Étapes du Repas (État)	3
2.3	Évolution 3 : Notification du Comptoir (Observateur)	4
2.4	Évolution 4 : Menus (Stratégie)	5
3	Code Java Intégral	6
3.1	Classe principale pour tester l'application	6
4	Schéma UML Intégral de la Solution	10
5	Conclusion	11

1 Introduction et Consignes

Ce document présente la solution de modélisation du Devoir 2, intégrant les évolutions du modèle de cantine à l'aide des quatre Design Patterns imposés : **État**, **Décorateur**, **Stratégie**, et **Observateur**. L'objectif est de produire un **schéma UML unique** et ce document justificatif, mettant en évidence l'application des patterns.

2 Justification des Design Patterns Appliqués

2.1 Évolution 1 : Événements (Décorateur)

Pattern Décorateur

Problème : Modifier la décoration, les tables et les chaises pour divers événements (Déjeuner pour tous, Anniversaire, Privatisation) de manière dynamique.

Pattern Choisi : **Décorateur** (*Decorator*)

Justification : Le Décorateur permet d'ajouter des responsabilités à l'objet Cantine de manière dynamique et sans modifier sa classe de base.

```
1 public interface IServiceRestauration {  
2     void decorer();  
3     String getDescription();  
4 }
```

Listing 1 – Interface de base pour la restauration

```
1 public abstract class DecorateurEvenement implements  
   IServiceRestauration {  
2     protected IServiceRestauration cantine;  
3  
4     public DecorateurEvenement(IServiceRestauration cantine) {  
5         this.cantine = cantine;  
6     }  
7  
8     @Override  
9     public String getDescription() {  
10         return cantine.getDescription();  
11     }  
12 }
```

Listing 2 – Classe abstraite du décorateur

```
1 public class AnniversaireEnfant extends DecorateurEvenement {  
2     public AnniversaireEnfant(IServiceRestauration cantine) {  
3         super(cantine);  
4     }  
5  
6     @Override  
7     public void decorer() {  
8         System.out.println("Ajout de ballons de couleurs et de chapeaux  
9         en carton.");  
10    }  
11  
12    @Override  
13    public String getDescription() {  
        return super.getDescription() + " [Theme: Anniversaire Enfant]";  
    }  
}
```

2 JUSTIFICATION DES DESIGN PATTERNS APPLIQUÉS

```

14     }
15 }

```

Listing 3 – Exemple de décorateur concret : Anniversaire Enfant

2.2 Évolution 2 : Étapes du Repas (État)

Pattern État

Problème : Modéliser le cycle de vie du repas par les employés (Préparation Culinaire, Mise en place, Service, Fermeture & Nettoyage). L'ordre des actions dépend de l'état actuel.

Pattern Choisi : État (*State*)

Justification : L'État permet de formaliser le cycle de vie de la gestion du repas.

```

1 public interface IEtatRepas {
2     void preparer();
3     void mettreEnPlace();
4     void servir();
5     void fermerEtNettoyer();
6     IEtatRepas etatSuivant();
7 }

```

Listing 4 – Interface pour les états du repas

```

1 public class GestionRepas {
2     private IEtatRepas etatActuel;
3
4     public GestionRepas() {
5         this.etatActuel = new EtatPreparation(); // Etat initial
6     }
7
8     public void executerEtapeSuivante() {
9         etatActuel.executer(this); // L'état gere la transition
10        etatActuel = etatActuel.etatSuivant();
11    }
12 }

```

Listing 5 – Classe de contexte (Gestionnaire d'états)

```

1 public class EtatPreparation implements IEtatRepas {
2     @Override
3     public void preparer() {
4         System.out.println("Préparation culinaire en cours...");
5     }
6
7     @Override
8     public void mettreEnPlace() {
9         System.out.println("Mise en place des tables et des chaises.");
10    }
11
12    @Override
13    public void servir() {
14        System.out.println("Le service n'est pas encore prêt.");
15    }
16
17    @Override

```

2 JUSTIFICATION DES DESIGN PATTERNS APPLIQUÉS

```
18     public void fermerEtNettoyer() {  
19         System.out.println("La fermeture n'est pas encore pr te.");  
20     }  
21  
22     @Override  
23     public IEtatRepas etatSuivant() {  
24         return new EtatMiseEnPlace();  
25     }  
26 }
```

Listing 6 – Exemple d'état concret : Préparation

2.3 Évolution 3 : Notification du Comptoir (Observateur)

Pattern Observateur

Problème : Dès qu'une assiette est enlevée par un client, les préparés au comptoir doivent être informés pour en mettre une nouvelle.

Pattern Choisi : Observateur (*Observer*)

Justification : Ce pattern gère la dépendance un-à-plusieurs entre le comptoir et les employés.

```
1 import java.util.ArrayList;  
2 import java.util.List;  
3  
4 public class Comptoir {  
5     private List<IObservateurComptoir> observateurs = new ArrayList<>();  
6     private int stockAssiettes = 10;  
7  
8     public void ajouterObservateur(IObservateurComptoir obs) {  
9         observateurs.add(obs);  
10    }  
11  
12    public void retirerAssiette() {  
13        stockAssiettes--;  
14        System.out.println("Une assiette a ete retiree. Stock actuel : "  
15            + stockAssiettes);  
16        notifierObservateurs();  
17    }  
18  
19    public void notifierObservateurs() {  
20        for (IObservateurComptoir obs : observateurs) {  
21            obs.mettreAJour(stockAssiettes);  
22        }  
23    }  
24 }
```

Listing 7 – Interface du Sujet (Comptoir)

```
1 public interface IObservateurComptoir {  
2     void mettreAJour(int stockRestant);  
3 }  
4  
5 public class PreposeComptoir implements IObservateurComptoir {  
6     @Override  
7     public void mettreAJour(int stockRestant) {
```

2 JUSTIFICATION DES DESIGN PATTERNS APPLIQUÉS

```

8      System.out.println("[Pr pos ] Notification re ue : il faut
9      pr parer " + stockRestant + " nouvelles assiettes.");
10     }

```

Listing 8 – Interface et classe de l’Observateur (Préposé au comptoir)

2.4 Évolution 4 : Menus (Stratégie)

Pattern Stratégie

Problème : Modéliser la préparation de différents menus (Standard, Végétarien, Sans Porc) qui sont similaires mais comportent des spécificités.

Pattern Choisi : **Stratégie** (*Strategy*)

Justification : Il permet de définir une famille d’algorithmes interchangeables pour la préparation des menus.

```

1 public interface IStrategiePreparation {
2     void preparerMenu();
3     String getDescription();
4 }

```

Listing 9 – Interface de la stratégie de préparation

```

1 public class Menu {
2     private IStrategiePreparation strategie;
3
4     public void setStrategie(IStrategiePreparation strategie) {
5         this.strategie = strategie;
6     }
7
8     public void preparer() {
9         strategie.preparerMenu();
10    }
11 }

```

Listing 10 – Classe de contexte (Menu)

```

1 public class StrategiePreparationVegetarien implements
2     IStrategiePreparation {
3     @Override
4     public void preparerMenu() {
5         System.out.println("Preparation des plats vegetariens : sans
6         viande, sans produits laitiers.");
7     }
8
9     @Override
10    public String getDescription() {
11        return "Menu Vegetarien";
12    }
13 }

```

Listing 11 – Exemple de stratégie concrete : Menu Végétarien

3 Code Java Intégral

3.1 Classe principale pour tester l'application

```

1 public class Main {
2     public static void main(String[] args) {
3         System.out.println("
4             =====");
5         System.out.println("DEMONSTRATION COMPLETE DE LA CANTINE");
6         System.out.println("
7             =====");
8
9         // --- Creation de la cantine de base ---
10        IServiceRestauration cantine = new CantineDeBase();
11        System.out.println("Description initiale : " + cantine.
12            getDescription());
13
14        // --- Evolution 1 : Application du Pattern Decorateur ---
15        System.out.println("\n--- EVOLUTION 1 : DECORATEUR ---");
16        IServiceRestauration cantineAnniversaire = new
17            AnniversaireEnfant(cantine);
18        cantineAnniversaire.decorer();
19        System.out.println("Description apres d coration : " +
20            cantineAnniversaire.getDescription());
21
22        // --- Evolution 2 : Application du Pattern Etat ---
23        System.out.println("\n--- EVOLUTION 2 : ETAT ---");
24        GestionRepas gestionRepas = new GestionRepas();
25        System.out.println("DEbut du cycle du repas.");
26        gestionRepas.executerEtapeSuivante(); // Preparation -> Mise en
27            place
28        gestionRepas.executerEtapeSuivante(); // Mise en place ->
29            Service
30        gestionRepas.executerEtapeSuivante(); // Service -> Fermeture
31        gestionRepas.executerEtapeSuivante(); // Fin
32
33        // --- Evolution 3 : Application du Pattern Observateur ---
34        System.out.println("\n--- EVOLUTION 3 : OBSERVATEUR ---");
35        Comptoir comptoir = new Comptoir();
36        PreposeComptoir prepareur = new PreposeComptoir();
37        comptoir.ajouterObservateur(prepareur);
38
39        System.out.println("Retrait de 3 assiettes...");
40        comptoir.retirerAssiette();
41        comptoir.retirerAssiette();
42        comptoir.retirerAssiette(); // Le prepar ur sera notifie 3 fois
43
44        // --- Evolution 4 : Application du Pattern Strategie ---
45        System.out.println("\n--- EVOLUTION 4 : STRATEGIE ---");
46        Menu menuStandard = new Menu(new StrategiePreparationStandard())
47            ;
48        menuStandard.preparer();
49
50        Menu menuVegetarien = new Menu(new
51            StrategiePreparationVegetarien());
52        menuVegetarien.preparer();
53    }
54 }

```

Listing 12 – Classe Main pour tester l'ensemble des fonctionnalités

```
1 public class CantineDeBase implements IServiceRestauration {
2     @Override
3     public void decorer() {
4         System.out.println("La cantine est deja decoree de maniere
5             standard.");
6     }
7     @Override
8     public String getDescription() {
9         return "Cantine standard";
10    }
11 }
```

Listing 13 – Classe de base CantineDeBase

```
1 public class PrivatisationThematique extends DecorateurEvenement {
2     private String theme;
3
4     public PrivatisationThematique(IServiceRestauration cantine, String
5         theme) {
6         super(cantine);
7         this.theme = theme;
8     }
9     @Override
10    public void decorer() {
11        System.out.println("Application du theme : " + theme);
12    }
13
14    @Override
15    public String getDescription() {
16        return super.getDescription() + " [Theme: " + theme + " ]";
17    }
18 }
```

Listing 14 – Classe de decorateur pour la Privatisation Thematique

```
1 public class EtatMiseEnPlace implements IEtatRepas {
2     @Override
3     public void preparer() {
4         System.out.println("La preparation est terminee.");
5     }
6
7     @Override
8     public void mettreEnPlace() {
9         System.out.println("Mise en place des tables et des chaises.");
10    }
11
12    @Override
13    public void servir() {
14        System.out.println("Service du repas en cours.");
15    }
16
17    @Override
18    public void fermerEtNettoyer() {
```

3 CODE JAVA INTÉGRAL

```
19         System.out.println("La fermeture et le nettoyage ne sont pas
20             encore pr ts.");
21     }
22     @Override
23     public IEtatRepas etatSuivant() {
24         return new EtatService();
25     }
26 }
27
28 public class EtatService implements IEtatRepas {
29     @Override
30     public void preparer() {
31         System.out.println("La preparation n'est plus necessaire.");
32     }
33
34     @Override
35     public void mettreEnPlace() {
36         System.out.println("La mise en place est deja faite.");
37     }
38
39     @Override
40     public void servir() {
41         System.out.println("Service des clients en cours.");
42     }
43
44     @Override
45     public void fermerEtNettoyer() {
46         System.out.println("Fermeture et nettoyage des locaux.");
47     }
48
49     @Override
50     public IEtatRepas etatSuivant() {
51         return new EtatFermeture();
52     }
53 }
54
55 public class EtatFermeture implements IEtatRepas {
56     @Override
57     public void preparer() {
58         System.out.println("Toutes les etapes sont terminees.");
59     }
60
61     @Override
62     public void mettreEnPlace() {
63         System.out.println("Toutes les etapes sont terminees.");
64     }
65
66     @Override
67     public void servir() {
68         System.out.println("Le service est termine.");
69     }
70
71     @Override
72     public void fermerEtNettoyer() {
73         System.out.println("Fermeture et nettoyage en cours.");
74     }
75 }
```

```
76     @Override
77     public IEtatRepas etatSuivant() {
78         return null; // Fin du cycle
79     }
80 }
```

Listing 15 – Autres etats du repas

```
1  public class StrategiePreparationStandard implements
   IStrategiePreparation {
2      @Override
3      public void preparerMenu() {
4          System.out.println("Preparation des plats standards.");
5      }
6
7      @Override
8      public String getDescription() {
9          return "Menu Standard";
10     }
11 }
12
13 public class StrategiePreparationSansPorc implements
   IStrategiePreparation {
14     @Override
15     public void preparerMenu() {
16         System.out.println("Preparation des plats sans porc.");
17     }
18
19     @Override
20     public String getDescription() {
21         return "Menu Sans Porc";
22     }
23 }
```

Listing 16 – Autres strategies de preparation

4 Schéma UML Intégral de la Solution

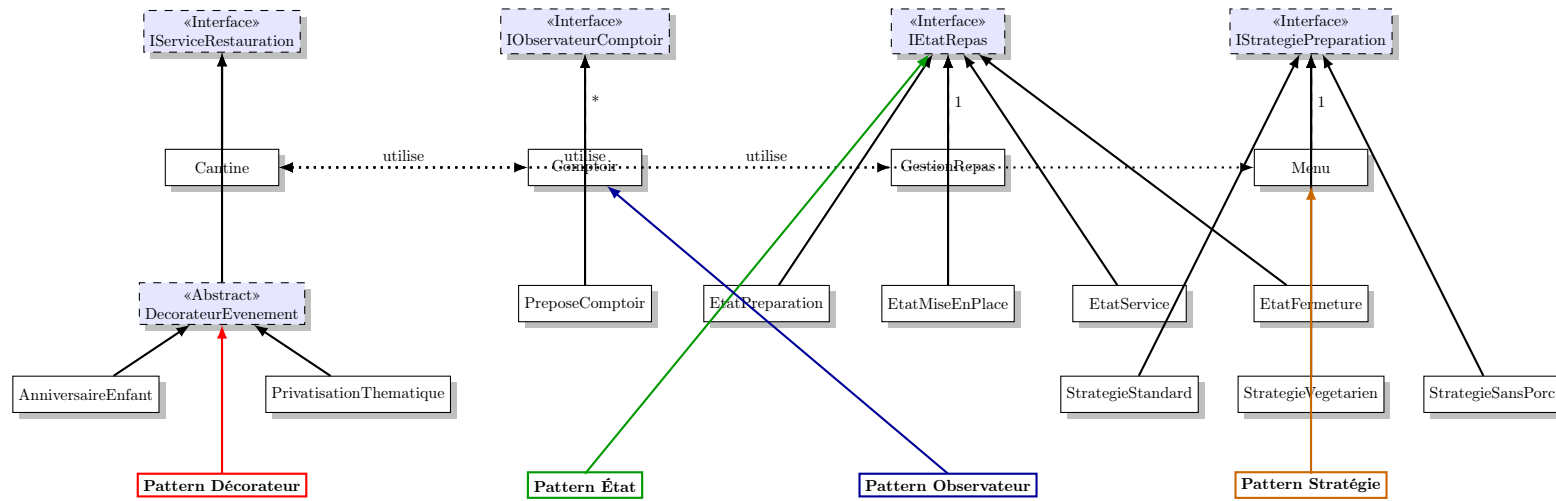


FIGURE 1 – Schéma UML intégré de la solution du Devoir 2 montrant l'application des quatre Design Patterns

5 Conclusion

La modélisation proposée pour la cantine intègre de manière cohérente les quatre Design Patterns imposés, chacun répondant à un besoin spécifique d'évolution du système :

- Le **Pattern Décorateur** permet d'ajouter dynamiquement des fonctionnalités événementielles sans modifier la classe de base.
- Le **Pattern État** formalise le cycle de vie du repas, rendant les transitions entre états claires et maintenables.
- Le **Pattern Observateur** assure une communication efficace entre le comptoir et les employés pour la gestion des stocks.
- Le **Pattern Stratégie** offre une solution flexible pour la préparation de différents types de menus.

Cette approche garantit une architecture **souple**, **extensible** et **maintenable**, respectant les principes SOLID de la conception orientée objet.